

COMPILER DESIGN

Phases of a Compiler and Their Role

NAME : PERARASAN V

REG NO : 192521404

COURSE CODE : CSA1405

Two Major Stages

Every compiler transforms source code into target code through Analysis and Synthesis

ANALYSIS · Front End

Breaks the source program into pieces and builds an intermediate representation, checking it is well-formed along the way.

Includes: Lexical, Syntax, and Semantic Analysis



SYNTHESIS · Back End

Constructs the target program from the intermediate representation, improving it and adapting it to the target machine.

Includes: Intermediate Code Generation, Optimization.

Front-End Phases

The analysis stage — understanding what the source program says

SCANNER

Lexical Analysis

- 1 Reads the stream of characters and groups them into meaningful tokens (keywords, identifiers, operators, literals), stripping out whitespace and comments.

PARSER

Syntax Analysis

- 2 Arranges tokens according to the grammar of the language, producing a parse tree / syntax tree that captures the program's structure.

TYPE & MEANING CHECKER

Semantic Analysis

- 3 Checks the syntax tree for semantic errors — type mismatches, undeclared variables, scope violations — using the symbol table.

Back-End Phases

The synthesis stage — turning meaning into efficient target code

IR BUILDER

Intermediate Code Generation

4

Translates the syntax tree into a low-level, machine-independent intermediate representation (e.g. three-address code).

OPTIMIZER

Code Optimization

5

Improves the intermediate code so the final program runs faster or uses less memory, without changing its meaning.

TARGET EMITTER

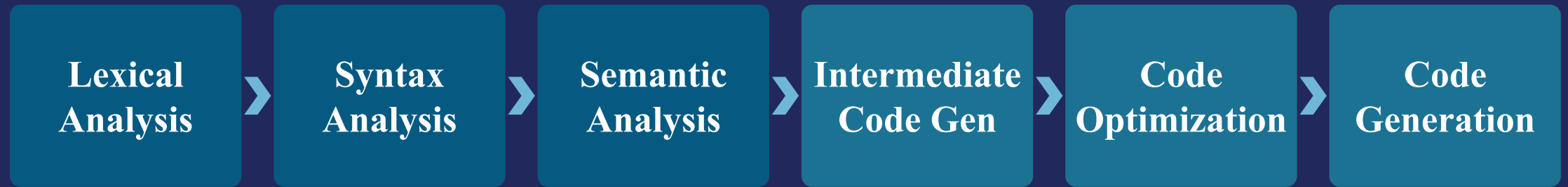
Code Generation

6

Maps the optimized intermediate code onto the target machine's instructions, assigning registers and memory locations.

The Complete Pipeline

Every phase feeds the next — errors and symbol-table entries flow throughout



SUPPORTED THROUGHOUT BY

Symbol Table Management

Records identifiers, types, and scope; consulted and updated by nearly every phase.

Error Handling

Detects and reports errors at each phase, and attempts recovery so compilation can continue.

Result: source code becomes correct, optimized, target-machine code.

The image features a soft, pink watercolor wash as a background. The wash is composed of various shades of pink, from light and airy to more saturated and vibrant tones, creating a textured, painterly effect. Overlaid on this background is the text "Thank You" in a black, elegant cursive script. The word "Thank" is positioned on the upper line, and "You" is on the lower line, with the two words connected by a fluid, sweeping line. The overall composition is centered and conveys a sense of warmth and appreciation.

Thank
You